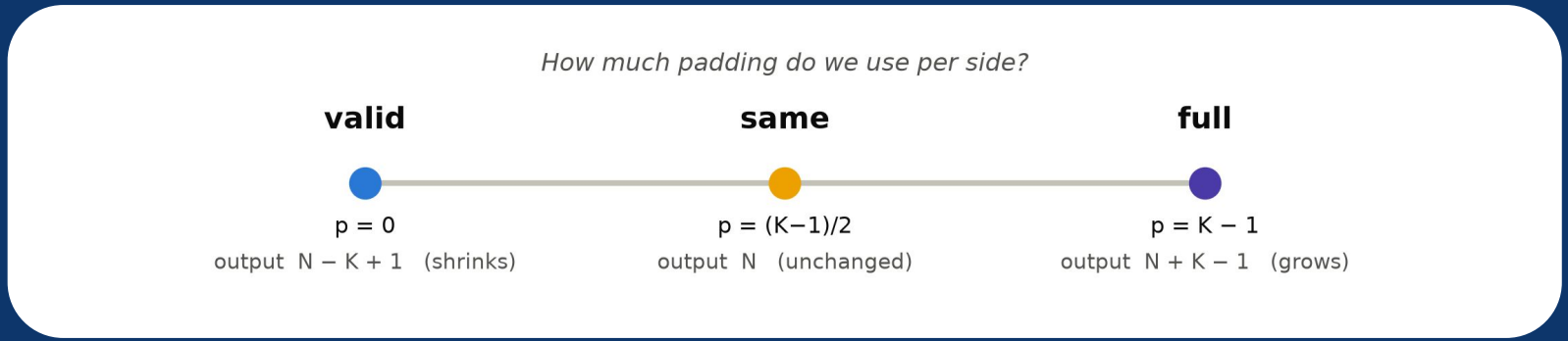
Convolution at the Edge

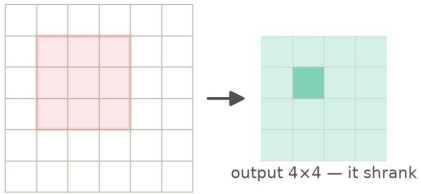
Valid, Same & Full Convolution

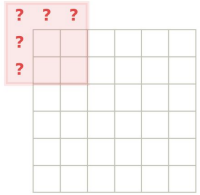
Wade Williams · CS 5330 Computer Vision



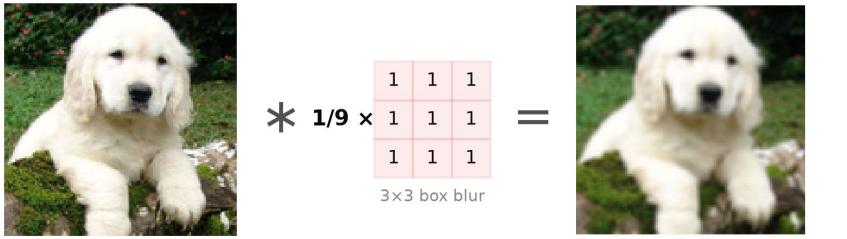
A familiar operation – convolution! But wait…the image shrinks?

By default, plain convolution shrinks the image!





- **Convolution:**
  1. Flip the kernel & slide the kernel over the image
  2. At each pixel, compute the dot product!
  3. Each stop makes one output pixel.
- **Example: 6x6 pixels + 3x3 Kernel —> only 4×4 came out**
- **Problem:** At the edges, the kernel hangs off the image — what multiplies the missing cells?



- **Real Example:** The same operation on a real photo – a 3x3 Gaussian blur filter.
- Puppy in: 128x128 · puppy out: 126x126
- We lose 2 pixels!

Problem 1 – The image shrinks!

When convolution is applied without padding, it leads to loss in spatial information and a smaller output image.

input: 108 × 200

after one 21x21 valid convolution pass: 88 x 180

N = image size (per dimension) K = kernel size

`output = N − K + 1  →  each pass: -(K-1)`

Here N = 108 x 200, K = 21 → every pass removes 20 pixels:

```
start    108 × 200
pass 1    88 × 180
pass 2    68 × 160
pass 3    48 × 140
pass 4    28 × 120
pass 5     8 × 100
pass 6     8 < 21 — kernel doesn't fit
```

(This applies to all kernels. A 3x3 loses 2 pixels per pass)

Problem 2 – We bias against the edges! (uneven usage of pixels)

When convolution is applied without padding, it leads to uneven usage of pixels across the image.

the original image (108 × 200)

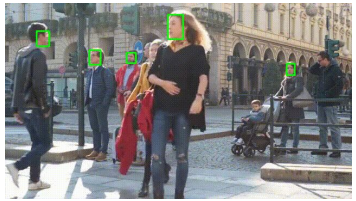
not an output — the input, re-lit by how often VALID (no-padding) convolution uses each pixel: brighter = used more (21×21 kernel)

dashed box: where the smaller 88 × 180 valid output sits

kernel placements that use each pixel (3×3 kernel, no padding)

1	2	3	3	2	1
2	4	6	6	4	2
3	6	9	9	6	3
3	6	9	9	6	3
2	4	6	6	4	2
1	2	3	3	2	1

EXAMPLE



a pedestrian entering the frame sits exactly where convolution listens least

- The kernel can only sit where it fully fits inside the image. The consequence of this is that the kernel passes over the edge pixels fewer times than it does the center pixels.
- **Thus, with no padding, the convolution extracts less information from the edges than it does from the center pixels.**

Solution: Padding! But how do we pad and what pixels do we keep?

Valid, same, and full are the three ways to handle the edges. Each method indicates how much padding we add, which decides where the kernel goes and how big the output is.

The solution to both problems: padding — add a ring of p extra pixels around the image before convolving.

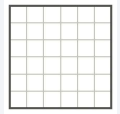
$$\text{output} = N + 2p - K + 1$$

Valid Convolution · p = 0

Valid Convolution is when convolution is computed only where the kernel lies entirely inside the image (no padding).

Output: N − K + 1 (shrinks)

Padding: none needed



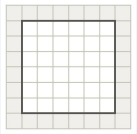
K = 3 ⟶ p = 0

Same Convolution · p = (K−1)/2

Same Convolution is convolution with just enough padding that the output is the same size as the input.

Output: N (unchanged)

Padding: any (e.g. zero, reflection)



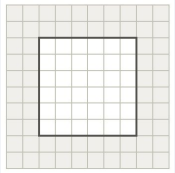
K = 3 ⟶ p = 1

Full Convolution · p = K − 1

Full Convolution is convolution computed at every position where the kernel overlaps the image at all.

Output: N + K − 1 (grows)

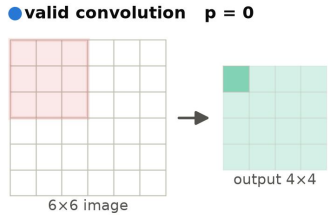
Padding: any (e.g. zero, reflection)



K = 3 ⟶ p = 2

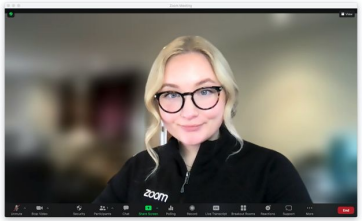
Valid Convolution – when would we use it? → 1 Possible Example: Zoom!

Recall valid convolution (no padding) shrinks the output image and biasing against the edges. Consequently, we might use valid convolution when we want to (1) reduce the spatial dimensions of our data, (2) avoid padding artifacts at the edges, or (3) ensure our kernel only operates on actual input pixels.



Valid padding applies convolution without adding any extra pixels, so the output becomes smaller than the input.

EXAMPLE — ZOOM BACKGROUND BLUR



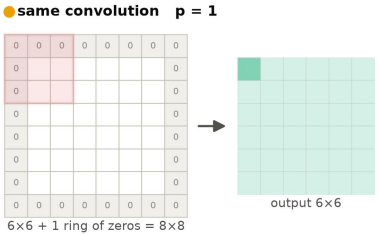
It's plausible that Zoom does valid convolution with a blur filter and no padding and displays a video stream that is slightly smaller and biased!

- **Advantages**
  - Nothing is made up — every output value comes from 100% real pixels
  - The output gets smaller — sometimes that's exactly what we want (less data to process)
- **Limitations**
  - We lose the borders — the output shrinks by K−1 every single pass
  - Edge pixels influence the output less
- **Other examples**
  - Template matching — match scores have to come from real pixels
  - Autofocus — sharpness scores from gradient statistics

MODE 2 OF 3 — SAME

Same Convolution – when would we use it? → 1 Example: Video games!

Same convolution (pad = (K-1)/2) keeps the output exactly the same size as the input. Consequently, we might use same convolution when we want (1) to composite results back onto the frame (2) chain filters without resizing, or (3) add and subtract filtered images from the original.



Same convolution ensures that the output has the same spatial dimensions as the input by padding just enough..

EXAMPLE — VIDEO GAME POST-PROCESSING

bloom = the glow around bright lights: blur the bright spots

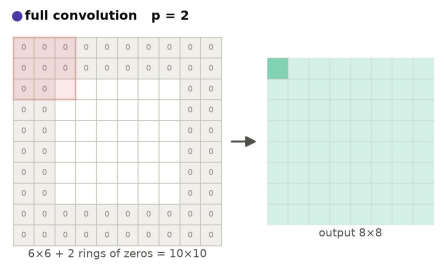
Every frame gets filtered and must come back exactly screen-sized – that's same convolution, 60 times a second!

- **Advantages**
  - The output is the same size as the input — so we can overlay, subtract, and chain freely
  - Every pixel gets an output, even the ones at the edges
  - Deep networks can stack layers without the image shrinking away
- **Limitations**
  - The border values are partly made up, so results near the edges can be subtly off
  - We compute a bit more — the kernel also slides over the padded ring
  - We have to choose what goes in the padding
- **Other examples**
  - Photo filters and unsharp masking — chains work because the size never changes
  - Detection and segmentation overlays that must land back on the frame
  - Most common in CNNs

MODE 3 OF 3 — FULL

Full convolution — when would we use it? Tracking at the frame edge!

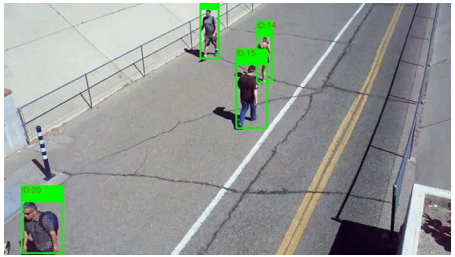
Full convolution grows the output and scores every overlap. Consequently, we might use full convolution when we want to (1) track objects that are only partly inside the frame, (2) keep a filter's complete response, or (3) build bigger kernels out of smaller ones.



For a K×K kernel: pad K−1 per side — even one pixel of overlap counts; output = N+K−1.

- **Advantages**
  - We keep every value the kernel can produce — the complete convolution
  - Every pixel gets used exactly K^2 times
- **Limitations**
  - The output grows by K−1 every pass — sizes drift if we chain filters
  - The outer band is computed mostly from padding, so it fades out
- **Other examples**
  - Photoshop glows & drop shadows — the effect spills past the layer, so the canvas grows
  - Building kernels from kernels

EXAMPLE — TRACKING AT THE FRAME EDGE



A pedestrian walks partly out of the frame and we still get a match score, because every overlap counts. Valid convolution might miss them.

Same image, same kernel — three different outputs

We apply the same kernel to the same image with each convolution mode, revealing the meaningful differences in the outputs based on how we handle the edges when doing the convolution.

Sobel-style edge detector (21×21 Gaussian derivative), Zero padding.

	input	valid	same	full
padding p	—	0	(K-1)/2 = 10	K-1 = 20
output size	—	88x180 (shrinks)	108x200 (unchanged)	128x220 (grows)
made-up pixels?	—	none	faint fake edges at the rim	a bright phantom frame
border usage	—	biased (1 … K^2)	covered, partly fake	equal – K^2 each
aligned?	—	no – shifted & cropped	yes – centered at same pixel	no – shifted

When to use which

What matters in our output and pipeline? Edge information? Consistent dimensions? Processing speed?

1. **Does the output need to be the same size as the input?** → SAME
2. **Does edge information matter? Do we need real data only?** → VALID
3. **Does every pixel matter?** → FULL

No strong yes? → same (the typical default)

mode	padding	output	advantages	typical use
valid	0	N-K+1	real data only — nothing invented	template matching · autofocus · Zoom
same	(K-1)/2	N	input & output dimensions are equal	default mode, video games, photo filters, overlays
full	K-1	N+K-1	no lost information	tracking at the edge · audio ring-out · glows

References & acknowledgements

References

- Dive Into Deep Learning (d2l.ai)
- “Padding in CNN Explained Simply” — youtube.com/watch?v=55Rc5sbHnc4
- “Convolution Padding – Neural Networks” — youtube.com/watch?v=ph4LrdntONo
- GeeksforGeeks: “CNN — Introduction to Padding”
- Euron: “CNN Padding: Valid vs. Same Explained!”
- DeepLearning.AI forum: “Padding to 0 — same vs valid”
- B. McFee, Digital Signals Theory — “Convolution Modes”

Code

- Code, slides & figures: GitHub repo — https://github.com/wade-g-williams/cs-5330-assessment-1

Valid vs. Same vs. Full Convolution – In Summary & When to Use?

Choosing whether to use valid, same, or full convolution and how to pad is largely application-dependent and is a design and engineering decision that makes a significant impact on the downstream effectiveness of the image processing pipeline. I.e. it matters!

	valid	same	full
	kernel fully inside — no padding	*pad just enough: output = input size*	*every pixel counts*
padding p	0	(K-1)/2	K-1
pad values	none needed	any	any
output size	N-K+1 (shrinks)	N (unchanged)	N+K-1 (grows)
aligned with input?	no	yes	no
advantages	100% real data, computation cost	consistent size, alignment	uses all pixels, all pixels equal (K^2)
limitations	shrinks · biased against edges	fabricated rim · must pick pad values	grows · outer band mostly padding
when to use	efficiency, don't need edge info., real data only	need consistent size, some info. loss is okay	all pixels matter!
examples	template matching · autofocus · Zoom	video games · photo filters · overlays	tracking at the edge · audio · glows